



UNIVERSIDAD AUTÓNOMA
DE AGUASCALIENTES

UNIVERSIDAD AUTÓNOMA DE AGUASCALIENTES
Centro de Ciencias Básicas

Sistema de Cifrado César y Atbash

con descifrado automático basado en el análisis de Al-Kindi

Alumno: Edson Leonardo Sánchez Montalvo
Grado y grupo: 7 A
Profesor: Arturo Ocampo Silva
Materia: Seguridad en Sistemas de Cómputo I
Proyecto: Sistema de Cifrado César y Atbash
Fecha: 9-9-2026

Índice

Sistema de Cifrado César y Atbash.....	1
1. Introducción.....	3
1.1 Contexto histórico.....	3
1.2 El cifrado César.....	3
1.3 El cifrado Atbash.....	3
1.4 ¿Por qué son cifrados simples?.....	3
1.5 Abu Yusuf Ya'qub ibn Ishaq al-Kindi.....	3
1.6 Cifrados clásicos frente a la criptografía moderna.....	4
2. Objetivo.....	4
3. Desarrollo.....	4
3.1 Arquitectura del sistema.....	4
3.2 Conjunto de caracteres configurable.....	5
3.3 Cifrado César.....	5
3.4 Cifrado Atbash.....	5
3.5 Descifrado automático.....	6
3.6 Análisis de frecuencia de Al-Kindi.....	6
3.7 Selección automática del resultado.....	6
3.8 Interfaz de usuario.....	6
3.9 Pruebas.....	6
4. Documentación segura y sistema XTZ.....	7
5. Publicación del sitio.....	9
6. Conclusión.....	10
7. Bibliografía.....	11

1. Introducción

1.1 Contexto histórico

La criptografía tiene sus orígenes en la necesidad de ocultar información durante conflictos militares y diplomáticos. Entre los métodos más antiguos y conocidos se encuentran el cifrado César y el cifrado Atbash. Ambos fueron diseñados para una época en la que la seguridad de la información dependía de la simplicidad del algoritmo y del secreto de quienes lo conocían.

1.2 El cifrado César

El cifrado César recibe su nombre del general romano Julio César, quien, según los registros históricos, lo utilizaba para proteger sus comunicaciones militares. El mecanismo es sencillo: cada letra del alfabeto se desplaza un número fijo de posiciones hacia adelante. Si el desplazamiento es 3, la A se convierte en D, la B en E, y así sucesivamente. Al llegar al final del alfabeto, la operación "da la vuelta" al inicio.

Es un cifrado de sustitución monoalfabética, lo que significa que cada letra del texto original se sustituye siempre por la misma letra en el texto cifrado, siguiendo una regla única y constante.

1.3 El cifrado Atbash

El cifrado Atbash tiene su origen en la tradición hebrea. Su funcionamiento consiste en invertir el alfabeto: la primera letra se intercambia con la última, la segunda con la penúltima, y así sucesivamente. Con el alfabeto latino, la A se convierte en Z, la B en Y, y la letra central, si existe, permanece en su lugar.

A diferencia de César, Atbash no requiere un número como clave, sino que tiene una única transformación posible. Además, es una operación simétrica: aplicar Atbash dos veces sobre un texto devuelve el texto original.

1.4 ¿Por qué son cifrados simples?

Ambos métodos son considerados cifrados simples por dos razones fundamentales:

El espacio de claves es extremadamente reducido. César tiene como máximo $n-1$ desplazamientos posibles, donde n es el tamaño del alfabeto. Para el alfabeto español de 26 letras, esto significa apenas 25 combinaciones. Atbash directamente no tiene clave.

Además, estos cifrados conservan la distribución de frecuencias del idioma original. Si la letra E es la más frecuente en español, en un texto cifrado con César seguirá apareciendo con la mayor frecuencia, aunque representada por otra letra. Esta propiedad permite romper ambos cifrados sin conocer la clave, aplicando análisis estadístico.

1.5 Abu Yusuf Ya'qub ibn Ishaq al-Kindi

Abu Yusuf Ya'qub ibn Ishaq al-Kindi (801–873 d.C.) fue un filósofo, matemático y científico árabe que vivió en Bagdad durante la Edad de Oro del islam. Es reconocido como uno de los primeros criptoanalistas de la historia.

En su tratado **Risāla fī Istikhrāj al-Mu'ammā** (*Manuscrito sobre el desciframiento de mensajes criptográficos*), escrito en el siglo IX, describió por primera vez el método de análisis de frecuencia (referencia consultada a través de Kahn [2] y Singh [3]). Su observación fue que, en cualquier idioma, ciertas letras aparecen con mayor regularidad que otras. En español, por ejemplo, la E aparece aproximadamente en el 13.7% de los textos, mientras que la Z aparece en menos del 1%.

Este descubrimiento permite romper cifrados de sustitución sin conocer la clave: si en un texto cifrado la letra más frecuente se repite muchas veces, es probable que corresponda a la letra más frecuente del idioma original. Comparando las frecuencias observadas con las esperadas, un analista puede determinar el desplazamiento probable.

1.6 Cifrados clásicos frente a la criptografía moderna

César y Atbash, aunque históricamente significativos, no son adecuados para proteger información sensible en el contexto actual. Un atacante con conocimientos básicos de criptoanálisis puede descifrar un mensaje en segundos.

La diferencia con la criptografía moderna es radical. Estándares como AES o RSA utilizan operaciones matemáticas computacionalmente irreversibles y claves de cientos de bits, lo que hace que romperlos por fuerza bruta sea computacionalmente inviable con la tecnología actual.

Este proyecto implementa César y Atbash como ejercicio académico para demostrar la comprensión de los principios criptográficos clásicos y la construcción de una interfaz web funcional.

2. Objetivo

Implementar un sistema web de cifrado y descifrado basado en los métodos César y Atbash, con capacidad de detección automática del algoritmo y desplazamiento utilizados, aplicando análisis estadístico inspirado en el trabajo de Al-Kindi.

Objetivos específicos:

1. Implementar el cifrado César con desplazamiento configurable sobre un conjunto de caracteres definido por el usuario.
2. Implementar el cifrado Atbash sobre un conjunto de caracteres definido por el usuario.
3. Permitir la configuración dinámica del conjunto de caracteres que participa en el cifrado.
4. Soportar cifrado y descifrado manual con ambos métodos.
5. Detectar automáticamente si el texto fue cifrado con César o con Atbash.
6. Detectar automáticamente el desplazamiento César utilizado.
7. Utilizar análisis estadístico de frecuencia inspirado en la metodología de Al-Kindi.
8. Mostrar automáticamente la solución descifrada correcta sin intervención del usuario.
9. Documentar el código fuente mediante el sistema de identificación técnica [XTZ-XX].
10. Publicar el sistema como aplicación web accesible.

3. Desarrollo

3.1 Arquitectura del sistema

El sistema está implementado como una aplicación web estática cliente-side. No requiere backend ni servidor; todo el procesamiento ocurre en el navegador del usuario. La estructura del proyecto se compone de tres archivos principales:

`index.html` contiene la estructura HTML semántica de la interfaz. Define todos los contenedores, campos de entrada, botones, área de resultado e indicadores de estado. Cada elemento interactivo tiene un identificador único que JavaScript utiliza para manipular el documento.

`styles.css` contiene los estilos visuales de la aplicación. Implementa un diseño claro y minimalista, tipografía moderna, microinteracciones y diseño responsive para diferentes tamaños de pantalla.

`script.js` contiene toda la lógica criptográfica, el sistema de análisis estadístico, el motor de pruebas y la gestión de eventos de la interfaz.

El flujo de procesamiento es el siguiente: el usuario ingresa texto en el área de entrada, el sistema valida el alfabeto configurado, selecciona el algoritmo según el modo elegido (cifrar, descifrar o automático), aplica la transformación correspondiente, evalúa estadísticamente los candidatos si el modo automático está activo, y finalmente presenta el resultado.

3.2 Conjunto de caracteres configurable

El sistema permite al usuario definir qué caracteres participarán en el cifrado. Esto significa que el alfabeto no es fijo, sino que puede ser cualquier conjunto de caracteres que el usuario defina.

La validación del alfabeto [XTZ-06] se realiza mediante la función $\times 06()$, que verifica tres condiciones: que el alfabeto no esté vacío, que tenga al menos dos caracteres, y que no contenga duplicados. Si alguna de estas condiciones no se cumple, el sistema muestra un mensaje de error descriptivo.

Los alfabetos predefinidos disponibles son el alfabeto básico latino (a-z, 26 caracteres), el alfabeto extendido con acentos y ñ (a-z más áéíóúüñ, 33 caracteres), el alfabeto ASCII (a-z más mayúsculas y dígitos, 62 caracteres) y el alfabeto ASCII de 7 bits (caracteres imprimibles 32–126, 95 caracteres).

Cualquier carácter que no pertenezca al alfabeto activo se preserva sin cambios durante la transformación. Esto incluye espacios, signos de puntuación, números no incluidos en el conjunto y caracteres Unicode siempre que no formen parte del alfabeto definido.

El sistema soporta cualquier carácter Unicode, no limitado a ASCII. Si el usuario define un alfabeto con caracteres acentuados o símbolos especiales, el sistema los procesa correctamente.

3.3 Cifrado César

El cifrado César [XTZ-07] transforma cada carácter del alfabeto desplazándolo k posiciones hacia adelante. La operación módulo garantiza que el desplazamiento "dé la vuelta" al inicio del alfabeto cuando se excede el límite.

La fórmula matemática es: $C(i) = (i + k) \bmod n$

Donde i es la posición del carácter en el alfabeto, k es el desplazamiento y n es el tamaño del alfabeto.

Para descifrar [XTZ-08], se aplica el desplazamiento inverso: $D(i) = (i - k) \bmod n$. En la implementación, $\times 08()$ reutiliza $\times 07()$ con desplazamiento negativo.

El algoritmo maneja correctamente los desplazamientos negativos y los desplazamientos mayores que el tamaño del alfabeto mediante la función $\times 11()$ [XTZ-11].

Ejemplo: con alfabeto "abcde" ($n=5$) y desplazamiento $k=2$:

Original	Índice	Cálculo	Resultado
a	0	$(0+2) \bmod 5 = 2$	c
b	1	$(1+2) \bmod 5 = 3$	d
c	2	$(2+2) \bmod 5 = 4$	e
d	3	$(3+2) \bmod 5 = 0$	a
e	4	$(4+2) \bmod 5 = 1$	b

"abcde" cifrado con $k=2$ produce "cdeab". Descifrar "cdeab" con $k=2$ devuelve "abcde".

3.4 Cifrado Atbash

El cifrado Atbash [XTZ-09] invierte el alfabeto respecto a su posición central. La fórmula es: $A(i) = n - 1 - i$. El primer carácter se intercambia con el último, el segundo con el penúltimo, y así sucesivamente.

La propiedad fundamental de Atbash es su simetría: aplicar Atbash dos veces devuelve el texto original. Por esto, $\times 09()$ y $\times 09i()$ comparten la misma lógica de inversión.

Ejemplo: con alfabeto "abcde" ($n=5$):

Original	Índice	Cálculo	Resultado
a	0	$5-1-0 = 4$	e

b	1	$5-1-1 = 3$	d
c	2	$5-1-2 = 2$	c
d	3	$5-1-3 = 1$	b
e	4	$5-1-4 = 0$	a

"abcde" produce "edcba". Aplicar Atbash sobre "edcba" devuelve "abcde".

3.5 Descifrado automático

El descifrado automático [XTZ-19] es la funcionalidad más compleja del sistema. Su objetivo es determinar automáticamente si un texto fue cifrado con César o con Atbash, y en caso de César, cuál fue el desplazamiento.

El proceso genera todos los candidatos posibles. Para César, genera un candidato por cada desplazamiento (de 0 a $n-1$). Para Atbash, genera un único candidato. Cada candidato se evalúa mediante puntuación combinada [XTZ-16] y se selecciona el de mayor puntuación.

3.6 Análisis de frecuencia de Al-Kindi

El sistema implementa el método de análisis de frecuencia descrito por Al-Kindi [XTZ-12].

Se utilizan cuatro conjuntos de datos estadísticos del español: frecuencias de letras individuales [XTZ-02] (E: 13.72%, A: 12.53%, O: 8.68%, etc.), bigramas comunes [XTZ-03] ("de", "la", "el", "en", "qu", etc.), trigramas comunes [XTZ-04] ("que", "los", "las", "del", etc.) y palabras frecuentes [XTZ-05] ("el", "la", "de", "en", "un", etc.). Estos valores son datos de referencia del español basados en el Corpus de Referencia del Español Actual (CREA) [4], implementados en script.js como datos de trabajo del prototipo académico.

El test chi-cuadrado [XTZ-13] mide la diferencia entre las frecuencias observadas en el texto candidato y las frecuencias esperadas en español. Un valor χ^2 pequeño indica que el texto candidato es probable que sea español válido.

3.7 Selección automática del resultado

La puntuación combinada [XTZ-16] evalúa cada candidato considerando tanto el análisis estadístico como el lingüístico:

$\text{score_final} = 0.4 \times \text{freqScore} + 0.6 \times \text{lingScore}$. freqScore se calcula como $1 / (1 + \chi^2)$. lingScore es una combinación de bigramas, trigramas y palabras comunes.

El sistema detecta cuando existe ambigüedad [XTZ-18]: si la diferencia entre los dos mejores candidatos es menor al 5%, emite una advertencia informativa.

El resultado mostrado es siempre una única línea descifrada: el candidato con mayor puntuación combinada. Si hay ambigüedad, se muestra el candidato ganador junto con la advertencia.

3.8 Interfaz de usuario

La interfaz se organiza en dos columnas. En el panel izquierdo se encuentran los controles de configuración: el conjunto de caracteres con botones de preset y visualización del alfabeto activo, y los modos de operación con selección de método, control de desplazamiento y visualización del mapeo.

En el panel derecho se encuentran el área de entrada de texto, el botón de procesamiento con indicadores de estado (idle, procesando, éxito, error), el área de resultado con opciones de copiar y limpiar, y la información del proceso que muestra método, desplazamiento, puntuación y análisis.

La sección de pruebas está en la zona inferior como elemento plegable.

3.9 Pruebas

El motor de pruebas integrado [XTZ-22] contiene siete casos que verifican los algoritmos. Las pruebas de César básico y wrap-around verifican el desplazamiento y el comportamiento en los bordes del alfabeto. La

prueba de Atbash verifica la inversión correcta del alfabeto. Las pruebas round-trip verifican que cifrar y descifrar devuelven el texto original con ambos métodos. Las pruebas de auto-detección verifican que el sistema identifica correctamente el método y el desplazamiento, tanto para César como para Atbash. La prueba de casos difíciles verifica el comportamiento con textos muy cortos y textos sin sentido lingüístico.

4. Documentación segura y sistema X TZ

El código fuente de script.js utiliza identificadores breves [XTZ-XX] que permiten referenciar cada componente funcional. Estos identificadores constituyen un sistema de trazabilidad técnica que vincula el código con la documentación.

Código	Componente	Ubicación	Descripción
[XTZ-01]	Estado global	script.js	Guarda el alfabeto activo, el desplazamiento por defecto, los alfabetos predefinidos (básico, extendido, ASCII, ASCII 7 bits) y las tablas del español con la lista de palabras comunes. Todo el programa lee y escribe este estado.
[XTZ-02]	Frecuencias español	script.js	Frecuencia relativa de cada letra en español (ej. E 13.72%, A 12.53%, O 8.68%). Es el modelo esperado contra el que x12() y x13() comparan cada candidato.
[XTZ-03]	Bigramas español	script.js	Bigramas comunes del español con su peso (ej. 'de', 'la', 'el', 'en', 'qu'). Los usa x14b() para puntuar a cada candidato.
[XTZ-04]	Trigramas español	script.js	Trigramas comunes del español (ej. 'que', 'los', 'las', 'del'). Los usa x14t() para puntuar a cada candidato.
[XTZ-05]	Palabras comunes	script.js	Palabras frecuentes del español. Las usa x14w() contando coincidencias de palabra completa en el texto.
[XTZ-06]	Validación alfabeto	script.js	'x06()': Fija el alfabeto activo tras validarlo con x06v(): no vacío, mínimo 2 caracteres y sin duplicados. Entrada: cadena del alfabeto. Salida: true/false y errores en la interfaz.
[XTZ-07]	Cifrado César	script.js	'x07()': Desplaza cada carácter k posiciones con módulo n; lo que no pertenece al alfabeto se conserva. Entrada: texto, desplazamiento y alfabeto. Devuelve {result, charsTransformed, errors}. Se apoya en xN(), xG(), xGM() y xSh().

Código	Componente	Ubicación	Descripción
--------	------------	-----------	-------------

[XTZ-08]	Descifrado César	script.js	`x08()`: Descifra reutilizando x07() con desplazamiento negativo. Misma entrada y salida que x07().
[XTZ-09]	Cifrado/Descifrado Atbash	script.js	`x09()/x09i()`: Sustituye cada carácter por su espejo (n-1-i). Sin clave y simétricas: aplicar dos veces devuelve el original. Se apoyan en xG() y xGM().
[XTZ-11]	Normalización shift	script.js	`x11()`: Reduce cualquier entero positivo o negativo al rango 0 a n-1 con módulo. Entrada: desplazamiento y n. Salida: entero normalizado.
[XTZ-12]	Análisis de frecuencia	script.js	`x12()`: Cuenta las apariciones de cada carácter y las divide entre el total. Entrada: texto y alfabeto. Salida: {freq, total}.
[XTZ-13]	Chi-cuadrado	script.js	`x13()`: Suma $(obs-esp)^2/esp$ por carácter. Entrada: frecuencias observadas, esperadas y alfabeto. Salida: número; 0 es coincidencia perfecta y los valores chicos indican español probable.
[XTZ-14]	Análisis lingüístico	script.js	`x14b()`, `x14t()`, `x14w()`: Cuentan bigramas, trigramas y palabras comunes en minúsculas. Entrada: texto y tablas. Salida: puntajes numéricos que usa x15().
[XTZ-15]	Puntuación lingüística	script.js	`x15()`: Combina los tres conteos con pesos 0.3/0.2/0.5. Entrada: texto y alfabeto. Salida: {bigramScore, trigramScore, wordScore, total}.
[XTZ-16]	Puntuación combinada	script.js	`x16()`: Junta frecuencia (x12()+x13() convertida a $1/(1+\chi^2)$) y lengua (x15() normalizada entre 10 con tope 1) con pesos 0.4/0.6. Salida: {chiSquared, freqScore, lingScore, combined}.
[XTZ-17]	Candidatos César	script.js	`x17()`: Genera un candidato por cada desplazamiento de 0 a n-1 usando x08(). Entrada: cifrado y alfabeto. Salida: arreglo de {text, method, shift, charsTransformed}.
[XTZ-18]	Detección de ambigüedad	script.js	`x18()`: Compara los dos mejores puntajes; si la diferencia relativa es menor a 5% marca ambigüedad. Entrada: candidatos ordenados. Salida: {ambiguous, gap, topScores}.

[XTZ-19]	Descifrado automático	script.js	`x19()`: Genera los candidatos de x17() más el de x20(), los puntúa con x16(), los ordena y devuelve {winner, allCandidates, ambiguity, isAmbiguous}.
[XTZ-20]	Candidato Atbash	script.js	`x20()`: Aplica x09i() una sola vez y devuelve el único candidato Atbash {text, method, shift nulo, charsTransformed}.
[XTZ-21]	Procesamiento principal	script.js	`x21()`: Lee el modo (uO()), el método (uM()) y el desplazamiento (uS()) de la interfaz, normaliza con x11() y despacha a x07()/x09(), x08()/x09i() o x19(). Sin parámetros: escribe el resultado en la página.
[XTZ-22]	Motor de pruebas	script.js	`x22`: Framework con rs(), lg(), ok(), eq() y rp(), más las pruebas v01 a v07 y vAll que verifican cifrado, bordes, round-trip y autodetección.
[XTZ-23]	Obtener modo	script.js	`uO()`: Lee el modo seleccionado en la interfaz (cifrar, descifrar o automático). Salida: cadena del modo.
[XTZ-24]	UI por modo	script.js	`dMU()`: Muestra u oculta el panel manual y cambia el botón según el modo, apoyándose en uO().
[XTZ-25]	UI por método	script.js	`dMM()`: Muestra las opciones de César o la nota de Atbash según el método, apoyándose en uM().
[XTZ-26]	Inicialización	script.js	`boot()`: Fija el alfabeto inicial, conecta todos los botones y deja la interfaz lista. Se ejecuta al cargar la página.

Consideraciones de seguridad: César y Atbash no proporcionan seguridad criptográfica moderna. Una aplicación web cliente-side no puede ocultar su código fuente; cualquier usuario puede inspeccionar el JavaScript del navegador. Los identificadores XTZ son documentación técnica, no un mecanismo criptográfico. No existen contraseñas, API keys ni credenciales en el proyecto.

5. Publicación del sitio

Repositorio: <https://github.com/EdsonLeonardoSM/Seguridad-Cripto>

El repositorio contiene los archivos del proyecto: index.html, styles.css, script.js y este documento como documentación principal.

Aplicación web: <https://edsonleonardosm.github.io/Seguridad-Cripto/>

El procesamiento es 100% cliente-side. Toda la transformación criptográfica ocurre en el navegador del usuario. Los datos nunca se envían a ningún servidor. No se requiere backend.

6. Conclusión

Este proyecto demuestra la implementación práctica de dos cifrados clásicos como herramienta educativa. A través del descifrado automático se evidenció cómo la combinación de análisis estadístico y lingüístico permite romper cifrados de sustitución sin conocer la clave.

Los métodos implementados, aunque históricamente significativos, son inadecuados para proteger información sensible en el contexto actual. Su espacio de claves insuficiente y su vulnerabilidad al análisis de frecuencia los hacen obsoletos frente a estándares criptográficos modernos.

La diferencia entre la criptografía clásica y la moderna radica en la solidez computacional: César puede ser roto manualmente en segundos, mientras que AES-256 requeriría más tiempo que la edad del universo para ser descifrado por fuerza bruta con la tecnología actual.

El objetivo del proyecto fue cumplido: se implementaron ambos métodos, se permitió la configuración del alfabeto, se soportó el cifrado y descifrado manual, y se construyó un sistema de descifrado automático que presenta la solución correcta sin intervención del usuario.

7. Bibliografía

- Kahn, David. *The Codebreakers: The Story of Secret Writing*. Macmillan Publishers, Nueva York, 1967. ISBN 978-0-671-65947-5. https://openlibrary.org/works/OL2543107W/The_codebreakers — Contexto histórico de los cifrados clásicos y de los orígenes del criptoanálisis (César, Atbash y el aporte de Al-Kindi).
 - Singh, Simon. *The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography*. Anchor Books, Nueva York, 1999. ISBN 978-0-385-49532-5. <https://simonsingh.net/books/the-code-book/> — Explicación del análisis de frecuencia aplicado en el descifrado automático del proyecto.
 - National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. FIPS PUB 197. U.S. Department of Commerce, 2001. <https://doi.org/10.6028/NIST.FIPS.197> — Referencia del estándar moderno que se usa como contraste frente a los cifrados clásicos del proyecto.
 - Real Academia Española. Banco de datos (CREA). *Corpus de referencia del español actual*. <https://corpus.rae.es/lfrecuencias.html> — Corpus de referencia para las frecuencias del español usadas como datos de trabajo en script.js.
-